

Day 1 — Anticipated Questions and Model Answers

Doubt clarification for AI accuracy engineering,
statistics and PoC diagnosis

Questions participants are likely to raise, with answers written the way you would say them in the room. Difficult and challenging questions are marked, with guidance on handling them without losing the room or overclaiming.

Enterprise AI Engineering Bootcamp · Companion to the Day 1 presentation and study material

A. The 95% Target — Expect These in the First Hour

These arrive early and often with an edge, because the target came from management. Handle them well and you have the room for five days.

Q Management has told us to reach 95% accuracy. Are you saying that is wrong?

Not wrong — incomplete. It is a reasonable business instinct expressed in the only unit most people know. The difficulty is that 95% does not say what to measure, and for some systems it is trivially achievable while meaning nothing. A maintenance model that never predicts a failure can be 99.9% accurate. What we will do today is convert that instinct into a specification you can engineer against: for your system, which metric, on what data, at what threshold, with what cost of being wrong.

Q So can we reach 95% or not?

For some of your systems, yes, on the right metric. For others the honest answer is that 95% on the metric that matters is not achievable with the data available, and pursuing it will waste a year. Which is which is exactly what the evaluation and diagnosis work tells you — and knowing that in week two rather than month twelve is worth a great deal. I would rather give you a defensible answer per system than a comfortable one.

Handling this. Do not promise 95% and do not dismiss it. The programme's commitment is a credible engineering pathway with defined metrics, not a guaranteed number. If pushed, the honest framing is: "the achievable number depends on data quality, ground truth availability and how the metric is defined — three things we will assess this week."

Q Our vendor quoted 98% accuracy for their model. Why can't we get that?

Ask them three questions: 98% on what dataset, with which metric, under what conditions. Benchmark numbers are almost always measured on curated public datasets under favourable conditions. That is not dishonest, but it does not transfer. The same model on your images, with your lighting, your defect taxonomy and your class balance produces a different number, and the only way to know it is to measure on your data.

Q Our board has seen demos elsewhere that look far better than ours. What are they doing differently?

Usually nothing you are not. Demos are selected — they show the cases that work. Yours would look equally good if you chose the ten examples to show. The difference between an impressive demo and a dependable system is precisely what this week covers, and organisations that appear to be far ahead are frequently at the same stage with better presentation. That is not cynicism; it is worth knowing before you benchmark yourselves against a stage performance.

B. Metrics and Evaluation

Q We do not have a labelled evaluation set. Where do we start?

Start small and expert-driven rather than large and automated. Fifty to a hundred carefully chosen examples with agreed correct answers is worth more than ten thousand auto-labelled ones. Pick them to cover the range: common cases, known failures, edge cases, one or two of each rare class. Have a subject-matter expert confirm each. That set becomes what you measure everything against for the next two years, so two or three days of someone's time is well spent.

Q How large does an evaluation set need to be?

It depends on the size of the difference you want to detect. Roughly: to detect a twenty-point improvement, around ninety items; ten points, around three hundred and sixty; five points, around fifteen hundred. Halving the effect you want to detect roughly quadruples the sample. Do the power calculation for the effect you care about — and if it produces a number you cannot reach, that is important to know before running the experiment, because it means the comparison cannot be settled with the data you have.

Q Can we use the model itself to generate the evaluation set?

For test *questions*, yes — model-generated candidates are a reasonable starting point and save time. For the *correct answers*, no. If the model defines what is correct, you are measuring self-consistency rather than correctness, and you will never detect a systematic error because the model makes the same one in both places. The standard compromise is to generate candidates automatically and have experts verify and correct them.

Q Our metric keeps changing as the business changes its mind. How do we handle that?

Fix the metric definition for the duration of an experiment and version it. If the definition changes, you have not improved the system — you have changed the measuring instrument, and the before and after numbers are no longer comparable. Record the definition alongside every result. When the business genuinely needs a different metric, re-baseline and be explicit that old numbers do not carry across.

Q Why not just use F1 for everything? It combines precision and recall.

Because F1 weights them equally and in your systems they are rarely equally important. A missed defect costing ₹40,000 and a false alarm costing ₹400 should not be balanced fifty-fifty. F1 is a reasonable single number for comparing models during development; it is a poor basis for a production decision. Use precision and recall separately, with the operating point set from cost — or F-beta if you want one number with a weighting you chose deliberately.

Q Should we use PR-AUC or ROC-AUC?

PR-AUC whenever the positive class is rare, which describes most defect and failure problems. ROC uses the total negative count as its denominator, so with a hundred thousand negatives even nine hundred false positives produce a false positive rate under one percent and the curve looks superb — while precision sits at nine percent and the inspector opening those cases finds almost all are fine. Both numbers are correct; only one describes the experience of using the system.

Q How often should we re-measure?

On every meaningful change, and on a schedule regardless of changes. The scheduled run catches drift, which by definition happens without anyone doing anything. Monthly is a reasonable default for industrial systems; more often if inputs change quickly. The key is automation — if re-measuring requires someone to remember, it will stop within two months.

C. Statistics and Hypothesis Testing

Q We are engineers, not statisticians. How much of this do we actually need?

Much less than a statistics course, and more than most teams currently use. Practically: state a hypothesis with a number in it, choose between about six standard tests, understand why sample size determines what you can detect, and report an effect size alongside a p-value. That is a day's learning and it prevents the two most expensive mistakes — shipping changes that did nothing, and discarding changes that worked. You do not need to derive anything.

Q What p-value threshold should we use?

0.05 is the convention and a reasonable default, but treat it as a decision rather than a law. What matters more is deciding it in advance and reporting effect size alongside. If a wrong "it worked" conclusion is expensive — you are about to rebuild a pipeline around it — use 0.01. If you are screening a dozen candidates cheaply, 0.10 with a confirmation step is defensible. State which you used and why.

Q Our result was significant but the improvement was tiny. Should we ship it?

Usually no, and this is exactly why effect size must be reported. Significance says the difference is real; it says nothing about whether it is worth the cost. A 0.3% improvement that adds a service dependency, latency and ongoing maintenance is a poor trade. Ask what the improvement is worth annually in business terms and compare that with the cost of carrying the change.

Q What if the result is not significant? Does that mean the change failed?

It means you did not find evidence that it helped — not evidence that it did not. The most common cause is an underpowered test: too few examples to detect the effect present. Before concluding the change was useless, check whether your sample size could have detected the improvement you hoped for. If not, the honest conclusion is "inconclusive", and the next step is a larger evaluation set rather than abandoning the idea.

Q Can we just run the experiment again if we do not like the result?

On fresh data, legitimately. Repeatedly on the same data until it gives the answer you want, no — each look increases the chance of a false positive, and if you look often enough almost anything becomes significant. If you want to check progress as data accumulates, use a sequential design that accounts for interim looks and fix the stopping rule in advance.

Framing if this gets philosophical. "These rules exist because we are all inclined to believe our own work helped. They are not statistical formality — they are a defence against a very human bias, and the more invested you are in a change, the more you need them."

Q Why a paired test rather than just comparing the two averages?

Because your evaluation items differ in difficulty. Some questions are genuinely hard and both systems fail; some are easy and both succeed. An unpaired comparison treats all that variation as noise, and the noise is frequently larger than the improvement you are looking for. A paired test compares item by item, so item difficulty cancels out and you can detect smaller real improvements with the same data.

Q When would we use McNemar rather than a paired t-test?

When the outcome per item is binary — right or wrong — rather than a continuous score. That is most model comparisons. McNemar looks only at the items where the two systems disagreed, because the ones they both got right or both got wrong carry no information about which is better. It is both the correct test and the more sensitive one for that situation.

Q All these tests have assumptions we cannot verify. What then?

Use a bootstrap or a permutation test. Neither assumes a distribution. The bootstrap gives you a confidence interval for any metric by resampling; the permutation test gives you a p-value by shuffling which system produced which result. Both cost only compute, both work for complex metrics like mean average precision, and both are easy to explain to a sceptical stakeholder. If you are ever unsure which test applies, these two will not mislead you.

Q We compared our change against fifteen metrics and two improved. Is that a result?

Not on its own. Testing fifteen metrics at a 0.05 threshold, you would expect roughly one to look significant by chance even if the change did nothing. Two is barely above what chance predicts. The fix is cheap and must happen before the experiment: name one primary metric, and treat everything else as secondary and exploratory. If you want to keep testing many, apply a false discovery rate correction.

D. Thresholds, Calibration and Human-in-the-Loop

Q Our business will not accept a system that sends work to humans. They want full automation.

Two different objections hide in that. If the concern is headcount, note that the review queue usually handles a small fraction of traffic — the automation is real, just not total. If the concern is that human review looks like failure, reframe it: every professional escalates cases outside their competence and we consider that good practice. The alternative is a system guessing confidently on the cases it understands least, which is exactly where the expensive mistakes live.

Q How do we choose where to put the confidence thresholds?

Work backwards from two constraints. First, cost asymmetry — plot expected cost against threshold and find the minimum. Second, human capacity — the uncertain band must produce a review volume your team can absorb. In practice you often set the upper threshold from cost and the lower from capacity, then check the resulting automation rate is worth having. If it is not, that signals model improvement rather than wider bands.

Q What if the model's confidence scores are meaningless?

Then check calibration before anything else, because the whole threshold design depends on it. Bucket predictions by stated confidence and measure how often each bucket is actually correct. If the 90% bucket is right 60% of the time, the model is over-confident and needs calibrating. Temperature scaling is one parameter, fitted on a validation split, and usually removes most of the problem. Uncalibrated confidence is worse than none, because it creates false assurance.

Q Is it not simpler to just set the threshold at 0.5?

It is simpler, and it is a business decision made accidentally by a software library default. 0.5 asserts that a false positive and a false negative cost exactly the same. In your systems they almost never do. Moving the threshold costs nothing, requires no retraining, and is frequently the largest single improvement available.

Q Can we ask a language model how confident it is?

You can, and the answer will be fluent and largely uninformative. Self-reported confidence from a language model correlates weakly with correctness. Token log-probabilities are somewhat better but still measure fluency more than factual correctness. For anything where confidence drives a decision, use an external signal — retrieval scores, agreement across multiple samples, or a separate verification step — rather than the model's own assessment of itself.

E. Data, Labels and Root Causes

Q Our labels come from maintenance records and they are unreliable. Can we still build anything?

Yes, with realistic framing. Noisy labels put a ceiling on supervised performance and no modelling raises that ceiling. Three routes: use unsupervised anomaly detection where labels are too weak for supervision; use the noisy labels for a coarser task they can support; or improve labels for a subset and measure how much performance rises, which tells you whether wider relabelling is worth funding. The important shift is to stop attributing the ceiling to the model.

Q Two of our inspectors genuinely disagree about borderline cases. What do we do?

Measure the disagreement first — it tells you the realistic ceiling. Then treat it as a definition problem rather than a people problem. Put them in a room with fifty borderline examples and write down the rule that separates the classes, with pictures. The output is a written taxonomy with visual examples of every borderline case. Re-label a subset against it and measure the change. Unglamorous work that frequently produces more improvement than any modelling change.

Q How do we know if we have leakage?

Three checks. Results that are surprisingly good are the strongest signal. Trace each feature back to when it becomes known in the real process and remove anything only available after the outcome. And test on a genuinely later time period than your training data — if performance drops sharply, something in the split was leaking. Time-based splitting is the single most effective preventive measure for time-dependent problems.

Q We keep collecting more data and performance does not improve. Why?

Almost certainly because the constraint is not volume. Run the failure taxonomy exercise before collecting anything more. Usually failures concentrate in two or three categories, and those categories are about label consistency, a missing feature, or a shift in conditions — none of which more of the same data addresses. Targeted collection of specific under-represented conditions does help; bulk collection rarely does.

Q How do we tell aleatoric from epistemic uncertainty in practice?

Train a small ensemble — three to five models with different seeds or data subsets. Where members disagree with each other, the uncertainty is epistemic and more data will help. Where every member is consistently unsure in the same way, it is aleatoric and more data will not. It is not a perfect separation but it is cheap and good enough to decide whether a data collection programme is worth funding.

Q Is data augmentation a legitimate way to get more data?

For genuine invariances, yes — if a defect is still a defect when the image is rotated or the brightness shifts, augmenting along those axes teaches the model something true. For anything else it adds noise. The test is whether the augmentation produces images that could plausibly occur in your production environment. Rotating an inspection image ninety degrees when the camera is fixed teaches an invariance that does not exist and can cost accuracy.

F. Practical and Organisational

Q This all takes time we do not have. What is the minimum version?

Three things, in order. Build a fifty-example evaluation set with expert answers. Classify a hundred real failures into categories and count them. Write one measurable hypothesis for the largest category and test it properly. That is about a week for one person, and it will tell you more about your system than the previous three months of tuning. Everything else in this programme elaborates those three steps.

Q Who should own the evaluation set — engineering or the business?

Jointly, with distinct roles. The business owns what "correct" means, because that is a domain judgement. Engineering owns the mechanics — sampling, versioning, running it, keeping it honest. The common failure is engineering writing the correct answers themselves, which quietly bakes in their assumptions about what the system should do.

Q Our PoC works well. Why do we need any of this?

A fair question, and the answer is: how do you know? If it works well on the cases you have tried, that is genuine but limited evidence. The specific risks are that it works on cases you thought to try and fails on those you did not, and that you have no way to detect it getting worse. The evaluation set converts "it seems to work" into a number you can defend and monitor. If your PoC really is strong, the measurement will show that — and then you have evidence rather than an impression.

Q How do we present all this to management without drowning them?

Three numbers and one chart. The primary metric with its confidence interval, the error budget position, and the cost of current errors in currency. The chart is the failure taxonomy ranked by cost. That conversation goes considerably better than a table of p-values, and it puts the discussion where it belongs — on what the errors cost and what fixing them is worth.

Q What if the labelling work we need has no budget?

Cost it against the alternative. If you can show that two weeks of expert relabelling produces the gain that six months of modelling has not, that is a business case rather than a request. The threshold and cost analysis from Module 1 helps here too, because it puts a currency figure on the errors you are currently absorbing. Most labelling budgets are refused because nobody quantified what the current error rate costs.

G. Difficult Questions — Handle With Care

These challenge the premise of the programme or put you on the spot. The answers below are deliberately honest rather than defensive — that is what works with an engineering audience.

Q Is this not just standard statistics we already learned at university?

Much of it is, and that is rather the point. The techniques are not new; what is missing in most AI teams is applying them to AI systems. Very few teams state a null hypothesis before changing a prompt, or calculate a sample size before comparing two retrieval strategies. The gap is not knowledge but practice. If your team already does all of this rigorously, say so and we will spend the time on the parts you do not.

Q Will following this process actually get us to production?

It gets you a defensible answer about whether production is achievable, and a measurable path if it is. That is a weaker claim than "yes" and a more useful one. Some of your systems will turn out to be limited by data quality or ground truth in ways no engineering discipline overcomes — and identifying those early is worth as much as improving the others, because it redirects effort that would have been spent regardless.

Q We used AI coding assistants to build these systems quickly. Does this slow us down?

It should not, and used well the two combine. The assistant makes building fast; the discipline makes you confident that what you built is better. The specific risk with rapid assisted development is generating a great deal of plausible code with no measurement of whether it improved anything — which is how systems become large and no better. Build the evaluation harness first, then let the assistant iterate against it. That is a genuinely fast loop.

Q Our management wants results in six weeks. Is this realistic?

It depends entirely on the starting point. If ground truth exists and the data is reasonable, measurable improvement in six weeks is realistic — threshold and retrieval changes often produce visible gains quickly. If there is no evaluation set and labels are inconsistent, six weeks gets you a trustworthy baseline and a clear plan, not a finished system. Either is a good outcome; the failure mode is promising the second and delivering neither. Report the honest version early.

Q What if we do all this and the system still is not good enough?

Then you will know why, specifically, and that is genuinely valuable. The failure taxonomy tells you whether it is data, labels, features or approach. Sometimes the answer is that the problem as scoped cannot be solved with what is available — and the right response is to rescope: a narrower task, a human-in-the-loop design, or a different problem. Discovering that after three weeks of measurement is a good outcome. After eighteen months it is not.

Q You keep saying "measure first". Is that not just delaying the actual work?

It feels like delay and it is not. Without measurement, every subsequent week of work produces changes you cannot evaluate — so you accumulate modifications with unknown effect, and eventually nobody can say which of them helped or whether the system is better than it was six months ago. The measurement is what makes the rest of the work compound rather than churn. Two days spent on it is recovered within the first fortnight.

Q Different teams here have very different problems. Is one approach really right for all of them?

The specific metrics differ substantially — a vision system and a retrieval system share almost no metrics. What transfers is the discipline: define what better means, measure honestly, diagnose before improving, state a testable hypothesis, prove it. That is why today is structured as a loop rather than a set of techniques. Days 2 to 5 then go deep on the specifics for each system type.

General principle for the hard questions. Do not defend the programme. Answer honestly, including where the honest answer is "it depends" or "you may find this is not solvable as scoped." An experienced engineering audience will trust you considerably more for it, and it makes the rest of the week easier.

H. Quick Reference — One-Line Answers

If asked	Say
Why not just report accuracy?	On imbalanced data it can be 99.9% and worthless.
Precision or recall?	Whichever error costs you more. A business decision.
PR-AUC or ROC-AUC?	PR-AUC whenever the positive class is rare.
How big an evaluation set?	Enough to detect the effect you care about. Halving the effect quadruples the sample.
Is p below 0.05 enough?	It says the effect is real. Effect size says whether it matters.
Result not significant?	Check the test was powered to detect it before concluding anything.
Which test for two models?	Paired. McNemar if the outcome is right/wrong per item.
Assumptions unverifiable?	Bootstrap or permutation test. No assumptions, any metric.
Where to set the threshold?	Where total expected cost is lowest. Rarely 0.5.
Confidence scores unreliable?	Check calibration first. Temperature scaling usually fixes most of it.
Should we collect more data?	Classify existing failures first. The answer usually changes.
Model ceiling?	Roughly where your human experts agree with each other.
Results look great offline?	Assume leakage until you have ruled it out.
Tested fifteen metrics, two improved?	That is roughly what chance predicts. Name one primary metric first.
When are we done?	When you are comfortably inside the error budget the business agreed.

Companion documents: **Day 1 Presentation** (67 slides with speaker notes) and **Day 1 Study Material** (concept explanations with analogies, worked examples and technical depth).